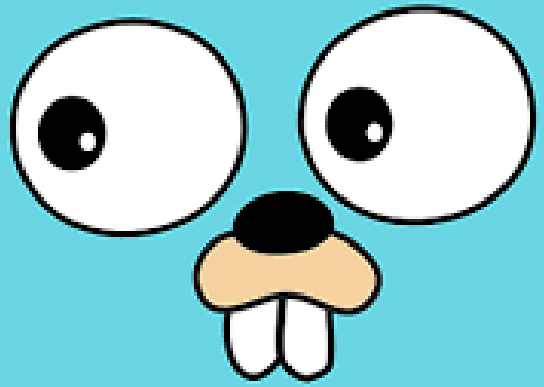


Concurrency in Go

Discussion Points

- Basic concepts of Concurrency
- Goroutines: creating and managing
- Channels: basic operations
- Buffered vs. unbuffered channels

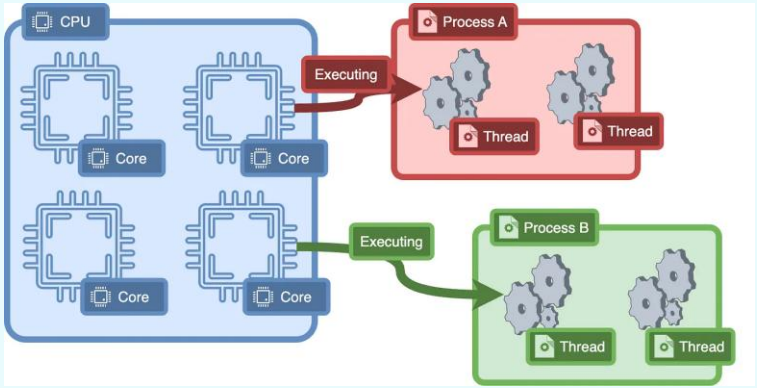
Basic concepts of Concurrency

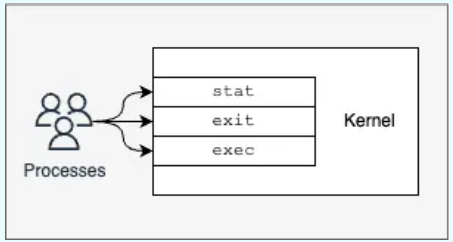
What is Process, Thread?

Nowadays, most computers are equipped with processors that have multiple cores, where each core functions as an independent processing unit that can operate largely on its own.

Threads are an abstraction built on top of these cores. In a program, we can create threads and run code on them, which the operating system's kernel can then schedule to execute on individual cores.

At any given moment, each core runs a single thread of execution.





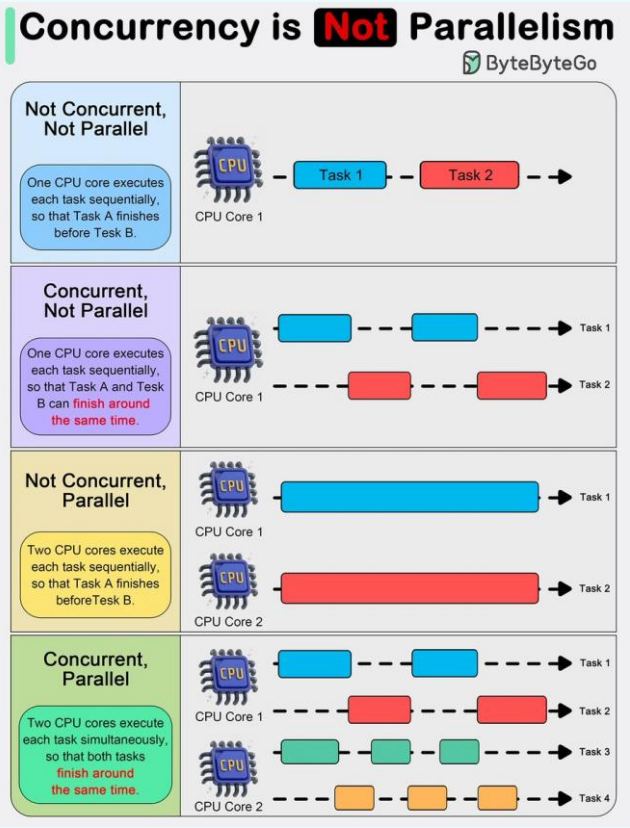
Basic concepts of Concurrency

Concurrency vs Parallelism?

Not the same, but related.

Concurrency is about dealing with lots of things at once

Parallelism is about doing a lot things at once

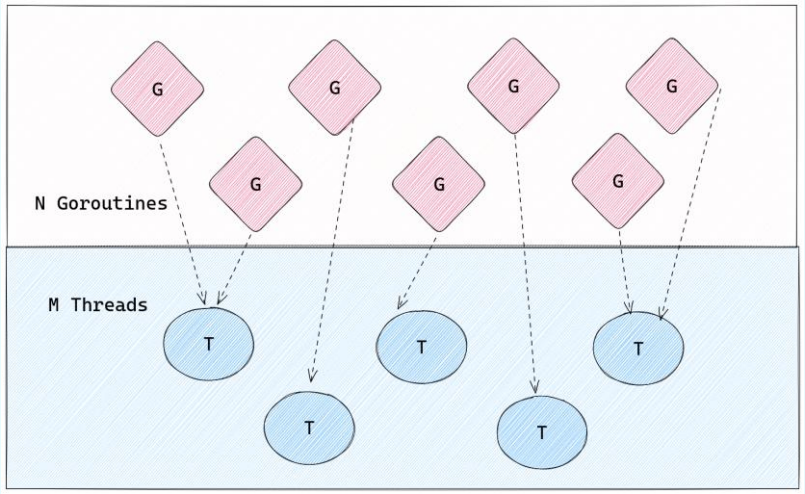


Goroutines: creating and managing

What is Goroutines?

Most primitive part of built-in concurrency in Golang; **light-weight (green) threads**

Go provides a special keyword **go** to create a goroutine. When we call a function or a method with **go** prefix, that function or method executes in a goroutine




```
go ping()
go pong()
```

Anonymous goroutines

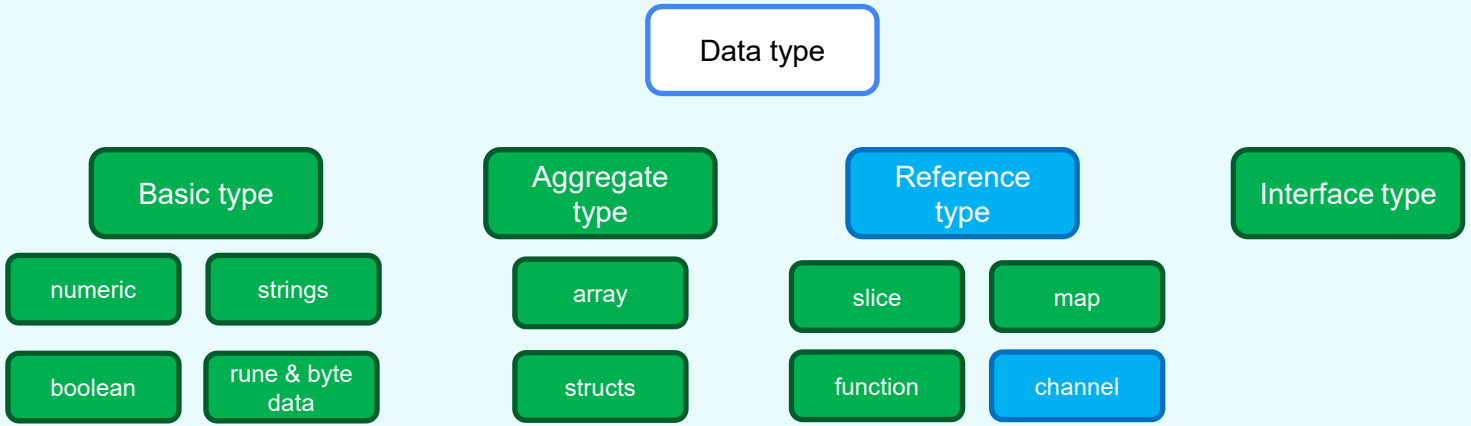
```
go func(){
    fmt.Println("Ping...")
}()
go func(){
    fmt.Println("Pong...")
}()
```

Goroutines: creating and managing

Threads vs Goroutines?

thread	goroutine
OS threads are managed by kernel and has hardware dependencies.	goroutines are managed by go runtime and has no hardware dependencies.
OS threads generally have fixed stack size of 1-2MB	goroutines typically have 8KB of stack size in newer versions of go
Stack size is determined during compile time and can not grow	Stack size of go is managed in run-time and can grow up to 1GB which is possible by allocating and freeing heap storage
There is no easy communication medium between threads. There is huge latency between inter-thread communication.	goroutine use channels to communicate with other goroutines with low latency
Threads have identity. There is TID which identifies each thread in a process.	goroutine do not have any identity. go implemented this because go does not have TLS(Thread Local Storage).
Threads have significant setup and teardown cost as a thread has to request lots of resources from OS and return once it's done.	goroutines are created and destroyed by the go's runtime. These operations are very cheap compared to threads as go runtime already maintain pool of threads for goroutines. In this case OS is not aware of goroutines.
Threads are preemptively scheduled. Switching cost between threads is high as scheduler needs to save/restore more than 50 registers and states. This can be quite significant when there is rapid switching between threads.	goroutines are coopertively scheduled. When a goroutine switch occurs, only 3 registers need to be saved or restored.

What is next with Data Types?



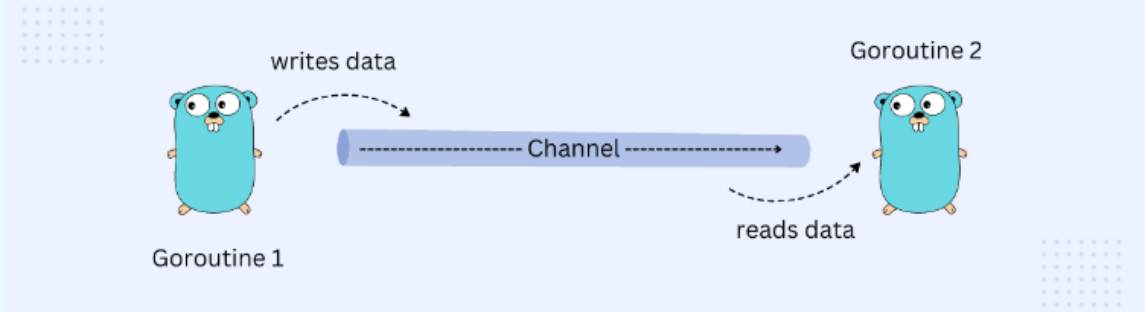
Channels: basic operations

Introduction?

Concurrency introduces challenges, such as managing access to shared resources where multiple threads can lead to inconsistencies.

While traditional languages use locks to safely share memory between threads.

Go's philosophy is to avoid shared memory and **instead share data through communication using channels**. This allows goroutines to pass data safely, ensuring only one goroutine accesses the data at any time.



Channels: basic operations

What is Channels?

A channel is a Golang data type that allows us to share data between goroutines. That makes it easier to write thread-safe concurrent programs and prevent race conditions.

Technically, a channel is a data transfer pipe where data can be passed into or read from. Hence, one goroutine can send data into a channel, while other goroutines can read that data from the same channel.

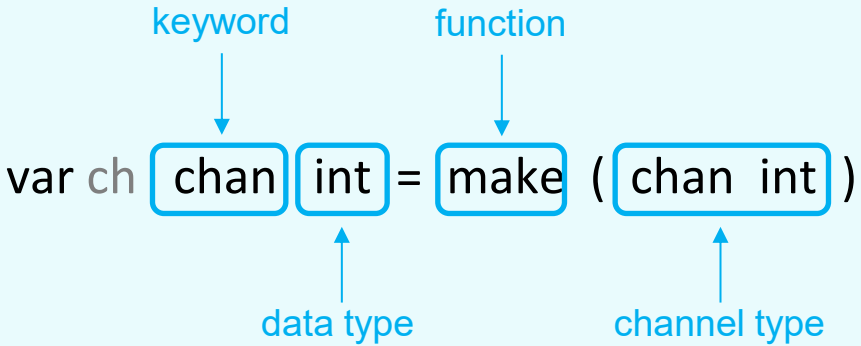
Channels can be summarized as:

- Channels facilitate safe communication and synchronization between goroutines without requiring explicit locks or condition variables.
- Channels can be sent as parameters to goroutines
- They work as both a publisher and subscriber model
- Internally, works like a FIFO circular queue
- Channels transfer copies of objects, rather than the actual objects

Channels: basic operations

Declaring a Channel?

Go provides **chan** keyword to create a channel. A channel can transport data of only one data type.



Buffered vs. unbuffered channels

Deadlocks in Channels?

When we write or read data from a channel, that goroutine is blocked and control is passed to available goroutines.

What if there are no other goroutines available, imagine all of them are sleeping. That's where deadlock error occurs crashing the whole program.

Seems like all goroutines are asleep or simply no other goroutines are available to schedule.

Channels: basic operations

Sending and receiving data in Channel?

Go provide very easy to remember **left arrow syntax <-** to read and write data from a channel.

write to int channel

read from int channel

```
ch <- 1

data := <- ch
```

When reading from or writing to a channel, if no value is available or no receiver is ready, the current goroutine blocks and the scheduler switches to another goroutine, ensuring the operation completes once a corresponding action occurs.

This built-in synchronization prevents blocking indefinitely and eliminates the need for manual locks in communication between goroutines.

Buffered vs. unbuffered channels

Types of Channels?

Based on how they store and handle data:

- **Unbuffered channels:** These channels have a capacity of 0 and require immediate synchronization between sender and receiver.
- **Buffered channels:** These channels can store multiple values up to a specified capacity.
- **Nil channels:** These channels are channels that are declared but not initialized.

Based on direction in which data can flow:

- **Bidirectional channels:** These channels allow both sending and receiving of values.
- **Unidirectional channels:**
  - Send-Only channels: These channels allow only sending values.
  - Receive-Only channels: These channels allow only receiving values.

Buffered vs. unbuffered channels

Unbuffered Channels (Synchronous Communication)?

Unbuffered channel, also known as *synchronous channel*, is the *default channel* type and has no capacity which means that only can handle one value at a time.

They ensure that data sent by the sender is immediately received by the receiver, leaving no room for delays. It's important to note that both sender and receiver must be ready before data can be sent or received. This allows goroutines to synchronize without explicit locks or condition variables.

By default, writing and reading from an unbuffered channel is blocking operation:

- A send operation will block until another goroutine receives from it.
- A receive operation will block until another goroutine sends to it.

In other words, an unbuffered channel is a way to transfer data between goroutines atomically.

Unbuffered Channels (Synchronous Communication)?

Advantages

Direct synchronization between sender and receiver. Ensures strict ordering of data exchange.

Disadvantages

Can lead to goroutine blocking if sender and receiver are not ready simultaneously. Can introduce potential deadlocks if not handled properly.

Buffered vs. unbuffered channels

Buffered Channels (Asynchronous Communication)?

Buffered channels, also known as asynchronous channels, provide greater flexibility by allowing a capacity to store multiple values.

Unlike unbuffered channels, they can hold data until a receiver is ready, enabling communication without immediate handoff.

For example, when multiple goroutines send messages, a buffered channel improves performance as it doesn't require each message to be processed immediately. It acts as a FIFO message queue, where the first message sent is the first to be received.

```
var ch = make ( chan int, n )
```

after chan type, we set buffer size

Buffered vs. unbuffered channels

Buffered Channels (Asynchronous Communication)?

Use Case
Buffered channels are useful when you want to decouple sender and receiver operations or when you need to reduce synchronization overhead. For example, you can use buffered channels to implement a worker pool, where multiple goroutines can send tasks to a limited number of workers for processing.

Advantages
Decouples sender and receiver operations.
Reduces synchronization overhead.

Disadvantages
Potential risk of goroutine leaks if not all data is consumed.
Can introduce some degree of latency due to buffering.

Points to consider:
Sends to a buffered channel block only when the buffer is full.
Receives block when the buffer is empty.
It’s crucial to select an appropriate buffer size. Too large a buffer might consume more memory, whereas too small a buffer might not optimize performance.

Channels: basic operations

Closing a Channel?

The Go garbage collector automatically collects channels that are no longer referenced, even if they contain buffered elements, without requiring them to be explicitly closed.

Channels in Go are not like resources that need closing to free memory, but closing them is significant for communication management, as it signals that no more values will be sent.

Sending and receiving data through channels is a one-to-one operation, meaning one goroutine sends while another receives. However, closing a channel acts as a broadcast, notifying all receiving goroutines simultaneously, making it the only way to communicate with multiple goroutines at once.

A send to a closed channel panics

```
close (ch)
```

A receive from a closed channel returns the zero value of the channel type

Channels: basic operations

Closing a Channel inside loop?

A general principle for using Go channels is: "Only close a channel from the sender side, and avoid closing it if there are multiple concurrent senders." This means that a channel should only be closed by the sender goroutine if it is the sole sender.

The receiver goroutine can determine the channel state using
v, ok := <- channel, where **ok** is *true* if the channel is *open*, and *false* if it is *closed*. This allows the receiver to know whether more read operations can be performed.

To further explore closed channel usability, let's look at using it in a *for loop*.

```
c := make(chan int, 3)
c <- 1
c <- 2
close(c)
for {
   if ch, ok := <-c; !ok {
      fmt.Println(ch, ok, "<-- loop broke!")
      break
   } else {
      fmt.Println(ch, ok)
   }
}
```

```
c := make(chan int, 3)
c <- 1
c <- 2
close(c)
for ch := range c {
   fmt.Println(ch)
}
```

Buffered vs. unbuffered channels

Bidirectional Channels?

As we know the channel is used to share data between goroutines and to do that the channel use directions (send-receive) by default Golang create the channel as a bidirectional mode which means it allows both sending and receiving of values. They don’t have any specific directional notation.

Use Case

Bidirectional channels are useful when you require two-way communication between goroutines, allowing them to send and receive data interchangeably. They provide flexibility in scenarios where both the sender and receiver need to interact.

Buffered vs. unbuffered channels

Unidirectional Channels?

A unidirectional channel is defined just to send or receive data.

```
var receiveOnly <-chan int // Can receive, cannot write or close
var sendOnly chan<- int    // Can send, cannot read or close
```

```
roch := make(<-chan int) // receive-only channel
soch := make(chan<- int) // send-only channel
```

Use Case

Send-only channels are useful when you want to send data from one goroutine to another without expecting a response or acknowledgement. For example, *you might use a send-only channel to send log messages to a logging goroutine for processing and writing to a log file.*

Receive-only channels are beneficial when you want to receive data from a channel without the need to send any data back. For example, *you might use a receive-only channel to receive notifications or updates from multiple goroutines and aggregate the results.*

Thank you.

Questions?

- Basic concepts of Concurrency
- Goroutines: creating and managing
- Channels: basic operations
- Buffered vs. unbuffered channels

“Learn from yesterday, live for today, hope for tomorrow. The important thing is not to stop questioning.”

- Albert Einstein